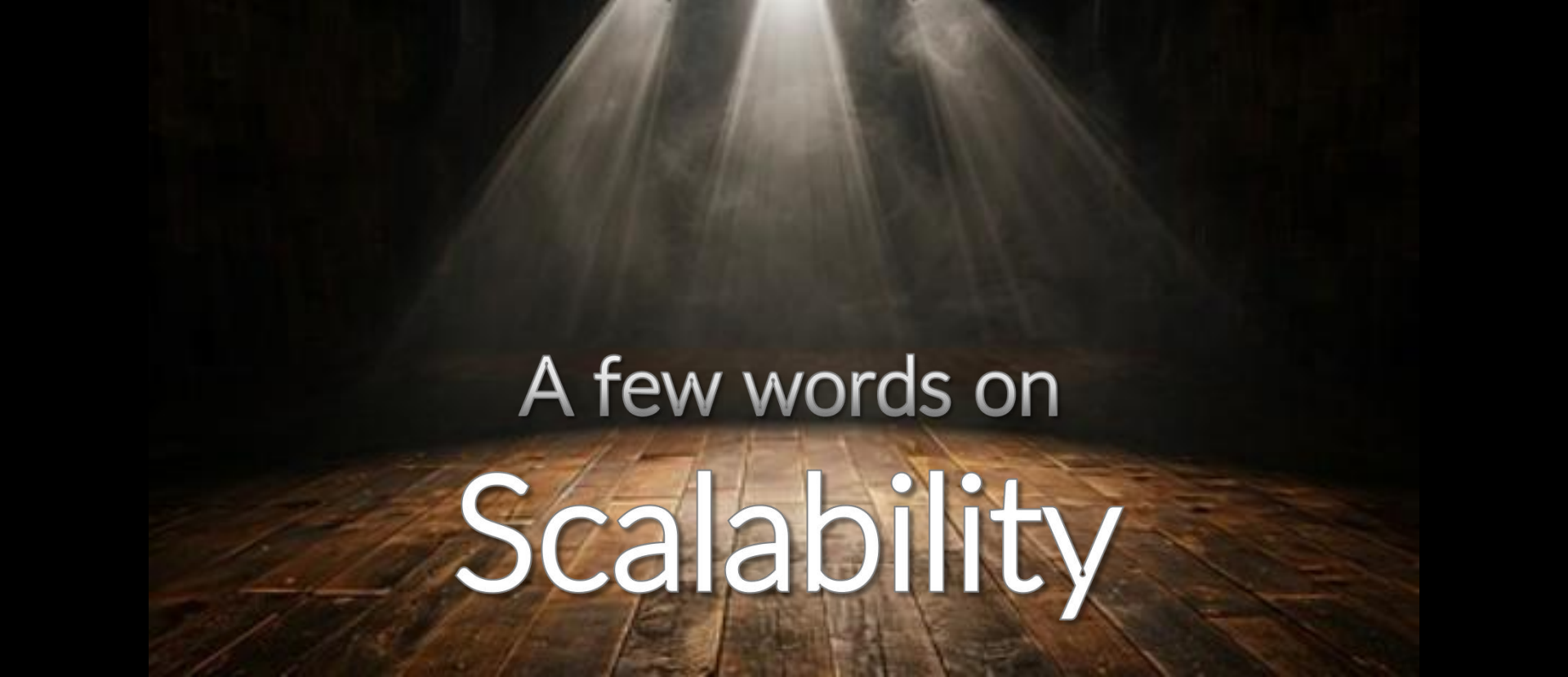




A few words on Scalability



Università di Trieste
HPC 2024-2025

Luca Tornatore
I.N.A.F.



| Scalability

I remind you that “scalability” is the capability of your code to efficiently use different amount of computational resources(*) when

1) the problem’s size is constant, which is called **strong scaling**

2) the problem’s size per resource is constant, which is called **weak scaling**

(*) “Resources” may be MPI tasks and/or threads (then basically how many cores you use) or nodes, if you always use entire nodes (which is oftne the case)

| Strong Scaling

Considering a given problem, we have seen in the lectures that we expect the time-to-solution to scale as $\propto 1/n$ if n is the amount of computational resources, for a perfect scaling.

The difference between the real scaling and the perfect scaling is due to either the intrinsic serial fraction f of the problem, or to parallel overhead k .

If your problem has a serial fraction, then your speed-up has an hard limit:

$$S_p = t(1, s) / t(n, s) \rightarrow 1/(1-f)$$

where $t(n, s)$ is the run-time for a size s , using n resources (which I remind you is called “the Amdahls’s law”).

| Strong Scaling

Then, when you test a code for its strong scaling capability, as first analyze your algorithm and individuate where you lose parallelism by its very nature (and, at the discussion, try to imagine a different algorithm or a way to express more parallelism).

Then, check whether your scaling results match with your analysis, and how the run-time scales once you purge the inherently serial part.

That is why requirement c) is about instrumenting your code so that you can check the timing of different sections *and* of the communications.

At the end, you should be able to characterize your code, and discuss whether any loss of parallelism is expected because it is due to the algorithm, or it is due to your implementation.

| Strong Scaling

Given that you expect a *linear* behaviour, the best way to present your results is *not* by plotting absolute times.

In fact, human eye is not good at judging non-linear behaviours, and in case you need to plot real timings, always superimpose the ideal scaling to your data.

The best way to report about scalability is to plot either the **Speedup**, as defined before and in the lectures and that it is expected to be linear, or the **efficiency**, which is defined as

$$\text{Eff}(n, s) = \text{Sp}(n, s) / n$$

which is expected to be ~ 1 for an ideal behaviour.

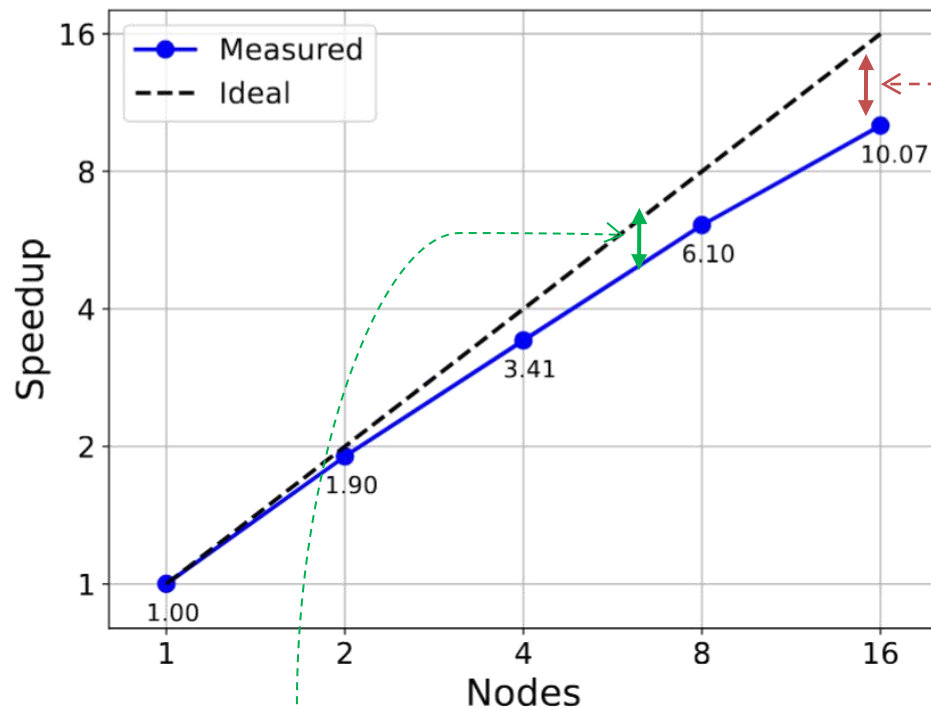
Strong Scaling

At the end, you end up with a plot that is similar to the one on the right.

on the x-axis there are the computational resources (in this case the nodes, not the MPI tasks or the threads), and on the y-axis there is speed up.

Note that the ideal scaling is reported to guide the eye.

There will always be some lost of efficiency for a large scaling factor, because for a large-enough amount of resources the parallel overhead will be dominant. There is not a formal definition of this threshold, since that is very much dependent on the problem and the algorithm. However I would say for good code that should happen beyond $16\times$...



gap due to inherent non-parallelism, communication, overhead due to inefficiency or to parallelization itself?

Strong Scaling

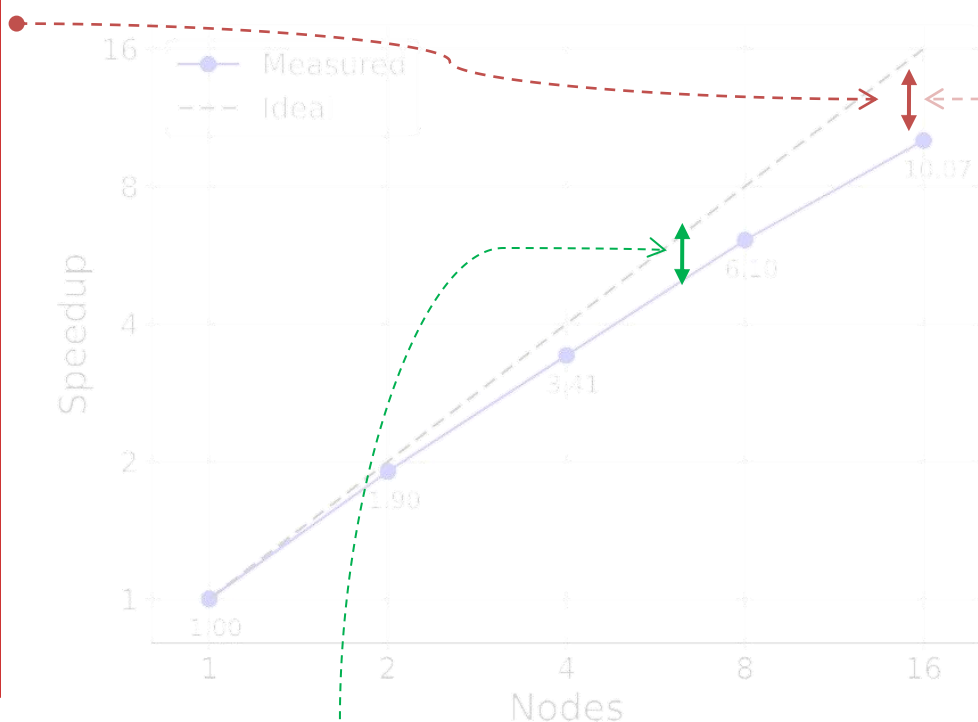
There will always be some loss of efficiency for a large scaling factor, because for a large-enough amount of resources the parallel overhead will be dominant. There is not a formal definition of this threshold, since that is very much dependent on the problem and the algorithm.

However I would say for good code that should happen beyond $16\times \dots$

It may be worth to stress that the problem's size must also be appropriate to scaling.

If that is too small since the beginning, your parallel overhead will be immediately dominant.

Getting what is “small” or “adequate” is obviously part of the exercise..



gap due to inherent non-parallelism,
communication, overhead due to inefficiency
or to parallelization itself?

| Weak Scaling

Weak Scaling measure how the time-to-solution varies when you grow the amount of resources n while keeping the workload per resource constant.

For instance, if the problem's size scales as N^2 , then the workload per resource is N^2/n .

Then, when you enlarge n , you have to keep this ratio constant to maintain the workload.

Reporting the weak scaling correctly is easier than for the strong scaling, since the expectation is that the run-time remains constant.

Then, also plotting the absolute timing is acceptable.

However, still, reporting the **efficiency** in this case give a better glance of how much the code loses (10%, 20%, 50% ?)

That`s all folks, have fun

“So long
and thanks
for all the fish”